

Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías
Análisis y Diseño de Algoritmos — 2026-1

Proyecto K-QGMIP

Manual de Usuario

Análisis de k -particiones de mínima información
Estrategias **KGeoMIP** y **KQNodes**

Guía de instalación y uso para usuarios finales

Autores

Milton Pablo Arias Rincon - 38790
Nelson Daniel Estrada Leiton - 9665

junio de 2026

Índice

1. Introducción y visión general	2
1.1. ¿Qué hace el software?	2
1.2. ¿Para qué sirve?	2
1.3. Conceptos básicos	2
1.4. Capacidades y limitaciones	2
2. Requisitos del sistema	2
3. Instalación paso a paso	3
4. Video tutorial de instalación y uso	3
5. Guía de uso básico	4
5.1. Preparación de los datos de entrada	4
5.2. Ejecución básica	4
5.3. Interpretación de resultados	4
5.4. Casos de uso típicos	5
6. Opciones y parámetros avanzados	5
6.1. Parámetros de configuración	5
6.2. Modos de operación	5
6.3. Opciones de salida y visualización	6
6.4. Optimización de rendimiento	6
7. Interfaz web (sin línea de comandos)	6
8. Solución de problemas	9
9. Ejemplos y tutoriales	9
9.1. Tutorial básico (4 nodos)	9
9.2. Caso intermedio (10 nodos, varios k)	9
9.3. Ejemplo avanzado (tabla completa)	9
10. Referencia rápida	10
10.1. Comandos principales	10
10.2. Glosario	10
10.3. Formato de archivos	10

1. Introducción y visión general

1.1. ¿Qué hace el software?

K-QGMIP analiza un **sistema** de variables binarias (descrito por una matriz de transición) y encuentra la mejor manera de **dividirlo en k partes** perdiendo la menor cantidad de información posible. En pocas palabras: responde a la pregunta “*si tuviera que romper este sistema en k pedazos, ¿por dónde lo cortaría para que el corte “duela” lo menos posible?*”.

1.2. ¿Para qué sirve?

Es una herramienta de **Teoría de la Información Integrada (IIT)**, usada para medir cuán “integrado” está un sistema (cuánto depende el todo de sus partes). Aplicaciones típicas: análisis de redes neuronales o genéticas, estudio de modularidad de sistemas dinámicos, y experimentación académica sobre IIT.

1.3. Conceptos básicos

- **Sistema:** conjunto de n nodos que pueden estar en estado 0 o 1 (binarios) o probabilidades continuas.
- **k -partición:** una división del sistema en exactamente k grupos (bloques). Con $k = 2$ es un solo corte; con $k = 3, 4, 5$ son cortes múltiples.
- **Partición de mínima información (MIP):** la división que menos información destruye. El número que mide ese “daño” se llama **pérdida** (símbolo ϕ o δ_k): *cuanto más pequeño, mejor partición.*

Idea clave: el programa prueba muchos cortes posibles y se queda con el que produce la *pérdida más baja*. Ese es el resultado que usted busca.

1.4. Capacidades y limitaciones

- **Sí puede:** analizar sistemas de hasta $n \approx 25$ nodos con las dos estrategias núcleo KGeoMIP y KQNodes, calculando la pérdida real δ_k para cualquier $k \in \{2, 3, 4, 5\}$. La tabla de evaluación oficial está resuelta al 100 % hasta N25A.
- **Con cuidado:** a $n = 25$ el subsistema completo tarda del orden de uno o dos minutos por fila y usa varios GB de RAM (pico $\approx 7,9$ GB en KGeoMIP), así que conviene una máquina con 16 GB.
- **No puede:** garantizar la partición óptima global en sistemas grandes (usa heurísticas voraces), ni manejar $n \gg 25$ en un equipo común.

2. Requisitos del sistema

Tabla 1: Requisitos mínimos y recomendados.

	Mínimo	Recomendado
Sistema operativo	Linux / macOS / Windows 10+	Linux (probado en Arch)
Python	3.14.5	3.14.5
RAM	4 GB ($n \leq 12$)	8 GB ($n \leq 20$); 16 GB para $n = 25$
Disco	500 MB	2 GB (incluye muestras grandes)
CPU	2 núcleos	4+ núcleos (paralelismo PCD)

Software necesario: el gestor de paquetes [uv](#) (instala Python y las dependencias automáticamente). Las bibliotecas (numpy, scipy, pandas, matplotlib, etc.) se declaran en `pyproject.toml` y se instalan con un solo comando.

3. Instalación paso a paso

Paso 1 — Obtener el código. Clonar el repositorio o descargar el `.zip`:

```
1 git clone <URL-del-repositorio> KQGMIP
2 cd KQGMIP
```

Paso 2 — Instalar uv (si aún no lo tiene):

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh # Linux / macOS
2 # Windows (PowerShell):
3 # powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Paso 3 — Instalar dependencias. Desde la carpeta del proyecto:

```
1 uv sync
```

Esto crea el entorno virtual y descarga todas las bibliotecas. No es necesario activar el entorno: cada comando se ejecuta con `uv run`.

Paso 4 — Verificar la instalación.

```
1 uv run pytest -q # debe terminar con todos los tests en verde
2 uv run exec.py # ejecuta un análisis de ejemplo
```

Nota. Si `uv run pytest` pasa, el software quedó correctamente instalado. El primer `uv sync` puede tardar varios minutos según la red.

4. Video tutorial de instalación y uso

Se incluye un video demostrativo que cubre la instalación desde cero, la configuración y un ejemplo completo de ejecución e interpretación.

Video tutorial: youtube.com/watch?v=NCtzkaL2t_Y

Contenido del video: (1) instalación de `uv` y `uv sync`; (2) ejecución desde la interfaz web y desde la CLI (`uv run exec.py run/batch/results/benchmark`); (3) ejecución con distintos valores de k ; (4) interpretación de la salida; (5) generación de figuras.

5. Guía de uso básico

5.1. Preparación de los datos de entrada

El sistema se describe con una **TPM** (matriz de transición) en un archivo CSV: `data/samples/N{n}{página}.csv`, con 2^n filas y n columnas de 0/1 o probabilidades continuas, en orden *little-endian* (la fila 0 es el estado 00...0). El programa la carga automáticamente según el número de nodos y la página. Si no tiene una TPM, puede generar una desde la **línea de comandos** con el script dedicado (no interactivo con `-yes`):

```
1 uv run scripts/generate_tpm.py --n 4 # determinista 0/1 (N4?.csv)
2 uv run scripts/generate_tpm.py --n 6 --continuous # probabilidades continuas
3 uv run scripts/generate_tpm.py --n 10 --seed 7 --yes
```

El archivo se guarda en `data/samples/` con el siguiente sufijo libre (A, B, ...). También puede generarlo desde la **interfaz web** (§7), sin línea de comandos.

5.2. Ejecución básica

El análisis individual se lanza con el subcomando `run` de `exec.py`, pasando los parámetros por línea de comandos (no hay que editar ningún archivo):

```
1 uv run exec.py run --net N10A --k 3 --strategy kgeomip
```

Opciones de `run`:

- `-net` red a analizar, p. ej. N10A (obligatorio).
- `-k` número de bloques, ≥ 2 (por defecto 3).
- `-strategy` `kgeomip`, `kqnodes`, `clustering`, `genetic`, `annealing`, `tabu` o `exhaustivek` (por defecto `kgeomip`).
- `-page` página de la red (A, B, ...); `-state` estado inicial binario; `-method` `spectral` | `kmeans` (solo `clustering`); `-condition/-purview/-mechanism` para fijar el subsistema a mano.

Sin argumentos, `uv run exec.py` imprime la ayuda y corre una demostración de extremo a extremo.

5.3. Interpretación de resultados

La salida muestra la estrategia usada, las distribuciones, la k -partición encontrada (bloques B1, B2, ...) y la **pérdida mínima** (φ). Ejemplo real (KGeoMIP, $k = 3$, red N10A):

```
1 KGeoMIP(k=3) fue la estrategia de solucion.
2 Distancia metrica: distancia-hamming
3 Notacion indexado: little-endian
4
5 Distribucion marginal del Subsistema:
6 [ 0. 1.0000 0. 0. 1.0000 1.0000 1.0000 0. 0. 0. ]
7 Distribucion marginal de la Particion:
8 [ 0. 1.0000 0. 0.4697 1.0000 1.0000 1.0000 0. 0.4727 0. ]
9
10 Mejor 3-Particion:
11 B1: abcdefghij | ABCEFGHJ
12 B2: ∅ | D
13 B3: ∅ | I
```

```

14 Perdida minima ( phi ) = 0.9424
15
16 Tiempos de ejecucion:
17 Horas: 0.00 = Minutos: 0.0 = Segundos: 0.0188

```

Cómo leerlo:

- **Bloques B1–B3:** cada bloque es presente | futuro. Las letras minúsculas (izquierda) son nodos de mecanismo/presente (t); las mayúsculas (derecha), de purview/efecto ($t+1$). Un lado vacío significa que ese bloque no aporta átomos de ese tiempo.
- **Pérdida mínima (ϕ):** 0,9424. Es el indicador clave: *más bajo es mejor*. Sirve para comparar estrategias o valores de k .
- **Tiempo:** cuánto tardó el análisis.

5.4. Casos de uso típicos

1. **Una 3-partición de un sistema pequeño:** `uv run exec.py run -net N10A -k 3 -strategy kgeomip`.
2. **Comparar $k = 2, 3, 4$:** repita cambiando `-k`; anote la pérdida de cada uno (crece con k).
3. **Comparar estrategias:** repita con `-strategy kqnodes` y con `-strategy clustering` sobre la misma red.

6. Opciones y parámetros avanzados

6.1. Parámetros de configuración

Tabla 2: Parámetros ajustables.

Parámetro	Bandera	Valores	Por defecto
k	<code>-k</code>	entero ≥ 2 (2–5)	3
estrategia	<code>-strategy</code>	kgeomip, kqnodes, clustering, exhaustivek, genetic, annealing, tabu	kgeomip
método	<code>-method</code>	spectral, kmeans (solo clustering)	spectral
página	<code>-page</code>	A, B, C, ...	la de <code>-net</code>
estado inicial	<code>-state</code>	cadena de 0/1 de longitud n	todo 1
semilla	<code>application.py</code>	entero	73

6.2. Modos de operación

- **Individual** (`uv run exec.py run -net ...`): un sistema, con los parámetros por línea de comandos.
- **Por lotes** (`uv run exec.py batch [datos.xlsx]`): recorre cada hoja `*-Elementos` del workbook, ejecuta KQNodes y KGeoMIP para $k \in \{2, 3, 4, 5\}$ y escribe la copia de resultados. Es reanudable y nunca modifica la entrada.
- **Ver resultados** (`uv run exec.py results [resultados.xlsx]`): imprime la tabla en la terminal (estrategia, k , pérdida, tiempo, partición).
- **Exacto** (`-strategy exhaustivek`): *ground truth* por enumeración; úselo solo en sistemas pequeños ($n \leq 6$).

6.3. Opciones de salida y visualización

```

1 uv run exec.py benchmark                # estrategias x redes x k -> CSV/Excel en
   data/results/
2 uv run exec.py benchmark --quick        # solo N10A (rapido)
3 uv run scripts/make_figures.py          # graficas estaticas (PNG): escalabilidad/
   perdida
4 uv run scripts/make_interactive.py      # graficas interactivas (HTML, Plotly)
5 uv run scripts/make_viz.py --net N4A --k 3 --strategy kgeomip # dibuja la
   particion

```

Las gráficas **estáticas** (PNG) quedan en `data/results/figures/` y las **interactivas** (HTML, navegables con *zoom* y *hover*) en `data/results/figures/interactive/`.

6.4. Optimización de rendimiento

- En sistemas pequeños `kgeomip` es más rápido; en sistemas grandes ($n \geq 22$) la relación se invierte y `kqnodes` resuelve más rápido (su preparación es más barata que la tabla de costos geométrica). Si el tiempo importa a n grande, prefiera `kqnodes`; ejecute ambas y tome el mínimo.
- Desactive el *profiling* para medir tiempos reales (`application.disable_profiling()`).
- El paralelismo por procesos se activa en las rutas que lo soportan; evite sobre-suscribir núcleos (el software ya fija `OMP_NUM_THREADS=1` en los trabajadores).

7. Interfaz web (sin línea de comandos)

Para quien prefiera no usar la línea de comandos, el proyecto incluye una **interfaz web** (Streamlit) que cubre todo el flujo desde el navegador. Streamlit y Plotly son dependencias de núcleo, de modo que `uv sync` ya las instala; basta con lanzar la aplicación:

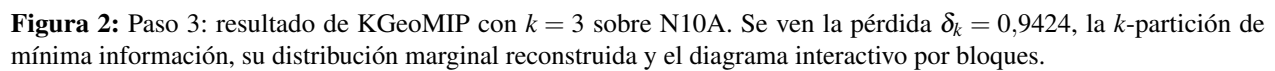
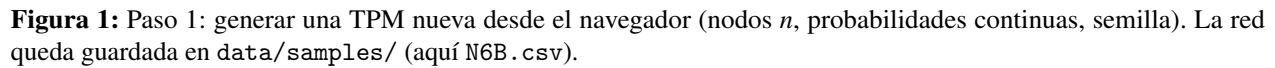
```

1 uv run streamlit_app.py

```

Se abre en `http://localhost:8501`. La interfaz tiene tres pasos:

1. **Datos.** Elija una muestra existente o *genere una TPM nueva* (nodos n , determinista o continua, semilla) con un botón; el CSV se crea en `data/samples/`.
2. **Estrategia.** Seleccione la estrategia (KGeoMIP, KQNodes, Clustering, Genética, Recocido, Tabú, ExhaustiveK), el número de bloques k y, si lo necesita, las máscaras de subsistema (condición, *purview*, mecanismo).
3. **Resultados.** Al pulsar *Ejecutar* verá la pérdida δ_k , la k -partición de mínima información, su distribución marginal en tabla, y el diagrama **interactivo** de la partición. Si existe la tabla de *benchmark*, se muestra además la gráfica interactiva δ_k vs k .



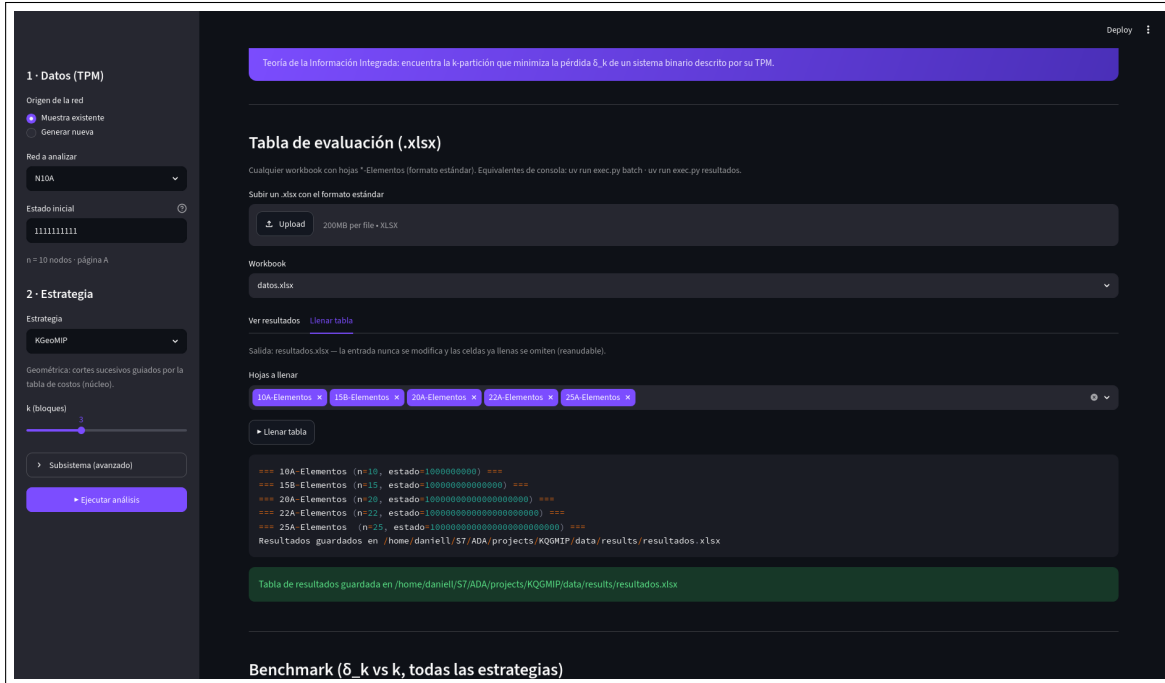


Figura 3: Tabla de evaluación (.xlsx) en la web: carga del workbook estándar, selección de hojas y vista de resultados tipo consola (estrategia, k , pérdida, tiempo, partición), equivalente a `uv run exec.py results`.

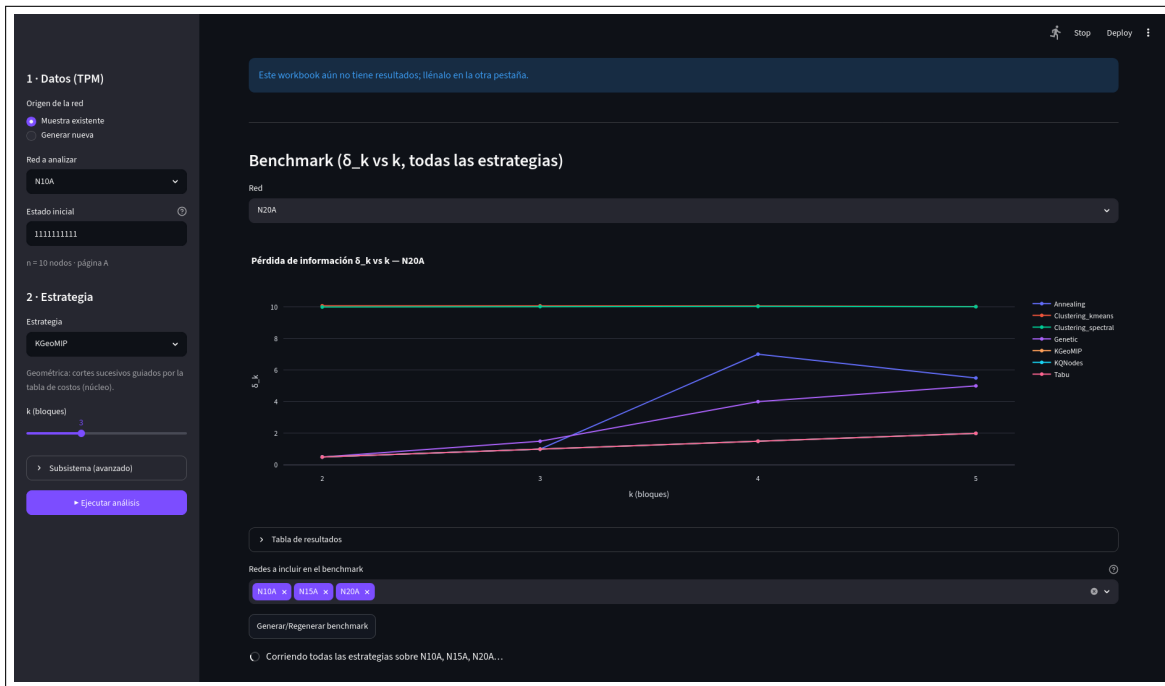


Figura 4: Benchmark δ_k vs k (N20A) comparando las siete estrategias. Abajo, el selector *Redes a incluir en el benchmark* permite elegir qué redes de `data/samples/` se ejecutan y regenerar el benchmark con ellas. Clustering espectral queda arriba (pérdida alta), mientras KGeoMIP, KQNodes y Tabú se mantienen en el mínimo.

8. Solución de problemas

Tabla 3: Errores comunes y solución.

Síntoma	Causa	Solución
FileNotFoundError: N{n}-{page}.csv	No existe la muestra para ese n /página	Generar la red o ajustar PAGE/initial_state
ValueError: k must be ≥ 2	$k < 2$	Usar $k \geq 2$
ValueError: strict k-partition ...	La partición dejaría un bloque vacío	Reducir k (no caben tantos bloques no vacíos)
MemoryError / proceso “muerto”	n demasiado grande (RAM)	Bajar n , o usar clustering
ModuleNotFoundError	Dependencias no instaladas	Ejecutar <code>uv sync</code>
ModuleNotFoundError: streamlit / plotly	Dependencias no instaladas	Ejecutar <code>uv sync</code>
Comando uv no encontrado	uv no instalado/PATH	Reinstalar uv y reabrir la terminal

Problemas de instalación. Si `uv sync` falla, verifique la conexión y la versión de Python (`uv python install 3.14`). En Windows, ejecute la terminal como usuario normal (no se requieren permisos de administrador).

Datos de entrada problemáticos. La TPM debe tener exactamente 2^n filas y n columnas con solo 0/1. Las cadenas `initial_state/conditions/purview/mechanism` deben tener longitud n .

9. Ejemplos y tutoriales

9.1. Tutorial básico (4 nodos)

1. Ejecutar `uv run exec.py run --net N4A --k 3 --strategy kgeomip`.
2. Leer la pérdida y los bloques. Dibujar la partición: `uv run scripts/make_viz.py --net N4A --k 3 --strategy kgeomip`.

9.2. Caso intermedio (10 nodos, varios k)

Con la red N10A, ejecute KGeoMIP para $k = 2, 3, 4$ y compare la pérdida (debe crecer): $0,4727 \rightarrow 0,9424 \rightarrow 1,4229$. Repita con `kqnodes` para confirmar que obtiene la misma calidad.

9.3. Ejemplo avanzado (tabla completa)

```

1 uv run exec.py batch                # llena la tabla de evaluacion (k=2..5)
2 uv run exec.py results              # ve la tabla en la terminal
3 uv run scripts/validate.py correctness # valida contra el exacto
4 uv run scripts/make_figures.py      # produce las graficas

```

Los resultados quedan en `data/results/` (CSV/Excel) y las figuras en `data/results/figures/`.

10. Referencia rápida

10.1. Comandos principales

Comando	Descripción
<code>uv sync</code>	Instalar dependencias
<code>uv run exec.py run --net N10A --k 3</code>	Análisis individual
<code>uv run exec.py batch</code>	Llenar la tabla de evaluación (Excel)
<code>uv run exec.py results</code>	Ver la tabla de resultados
<code>uv run exec.py benchmark</code>	Benchmark de estrategias
<code>uv run pytest</code>	Ejecutar los tests
<code>uv run scripts/make_viz.py</code>	Visualizar una partición

10.2. Glosario

TPM Matriz de probabilidad de transición que describe el sistema.

k -partición División del sistema en k bloques no vacíos.

Pérdida (φ / δ_k) Información destruida por una partición; menor es mejor.

KGeoMIP Estrategia geométrica (rápida, recomendada por defecto).

KQNodes Estrategia submodular (más robusta; más lenta en n pequeño, más rápida en $n \geq 22$).

Clustering Línea base rápida para sistemas grandes (solo propone).

Purview / Mechanism Conjuntos de nodos de efecto ($t+1$) y mecanismo (t) bajo análisis.

10.3. Formato de archivos

- **Entrada:** `data/samples/N{n}-{página}.csv` — 2^n filas, n columnas, 0/1, little-endian.
- **Salida:** resumen por consola; tablas en `data/results/*.csv/*.xlsx`; figuras PNG en `data/results/figures/`.